



openQSE – 5th Jan 2026

The Munich Quantum Software Stack (MQSS)

Lukas Burgholzer

lukas@munichquantum.software



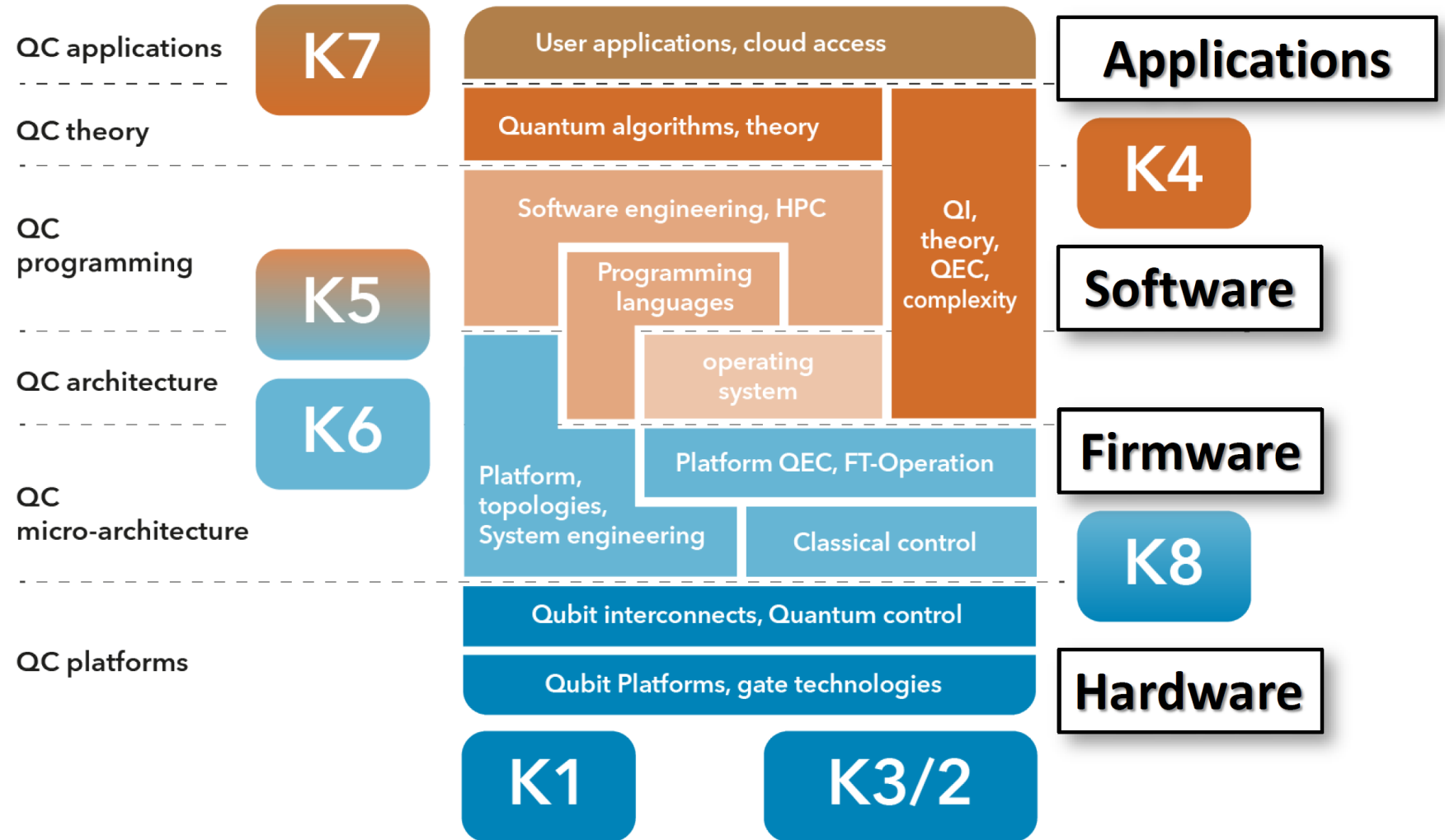


Munich Quantum Valley Institutions



The primary goal of the Munich Quantum Valley initiative is **developing and operating competitive quantum computers** in close cooperation with strong industry partners and visionary start-ups and making them available for a broad range of applications.

Munich Quantum Valley Structure



Munich Quantum Valley Platforms



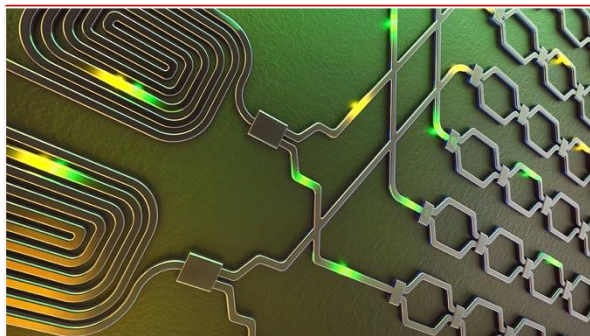
Trapped Ion

System 4: AQT 20
20+ qubits



Neutral Atom

System 7: PLQ 1000
1000 qubits (secured and coming)



Photonic

System X:
Searching for System



Superconducting

System 1: IQM 5
5 qubits

System 2: IQM 20 (1)
20 qubits

System 3: IQM 20 (2)
20 qubits (2024)

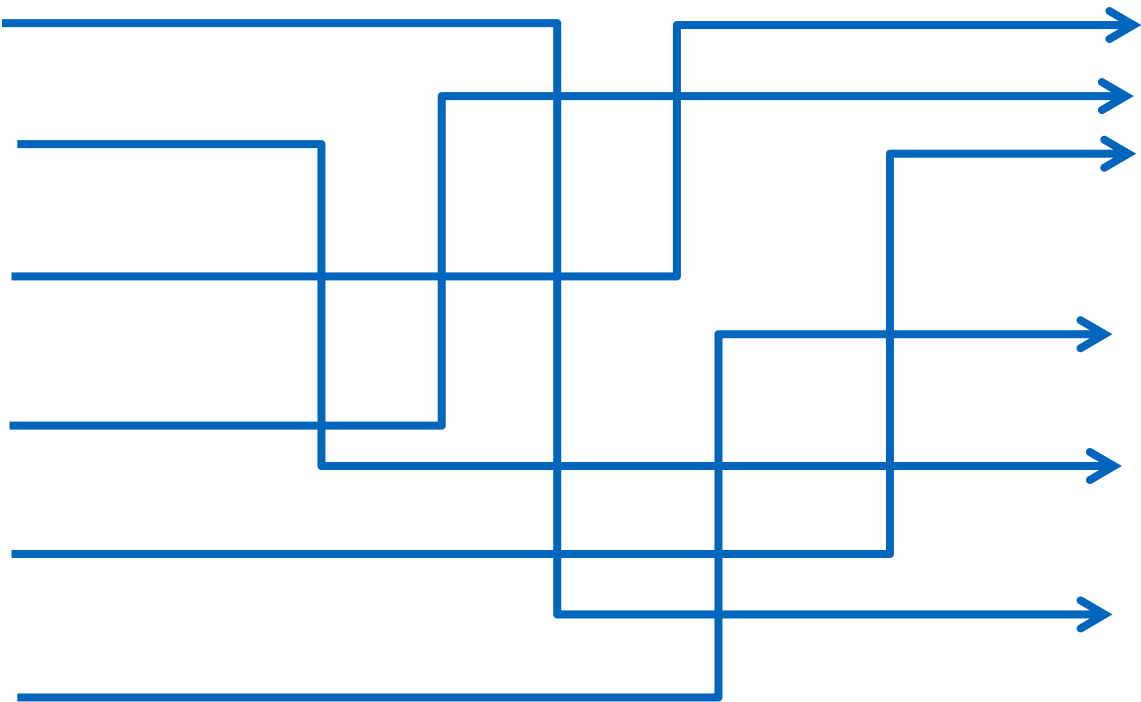


System 5: IQM 54
54 qubits (mid 2025)

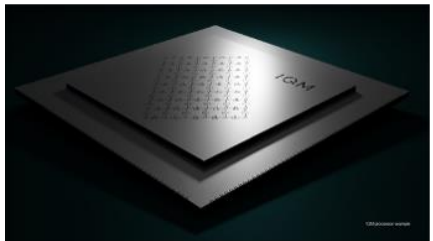
System 6: IQM 150
150 qubits (end 2026)

Quantum Computing – The Big Picture

Domain Experts

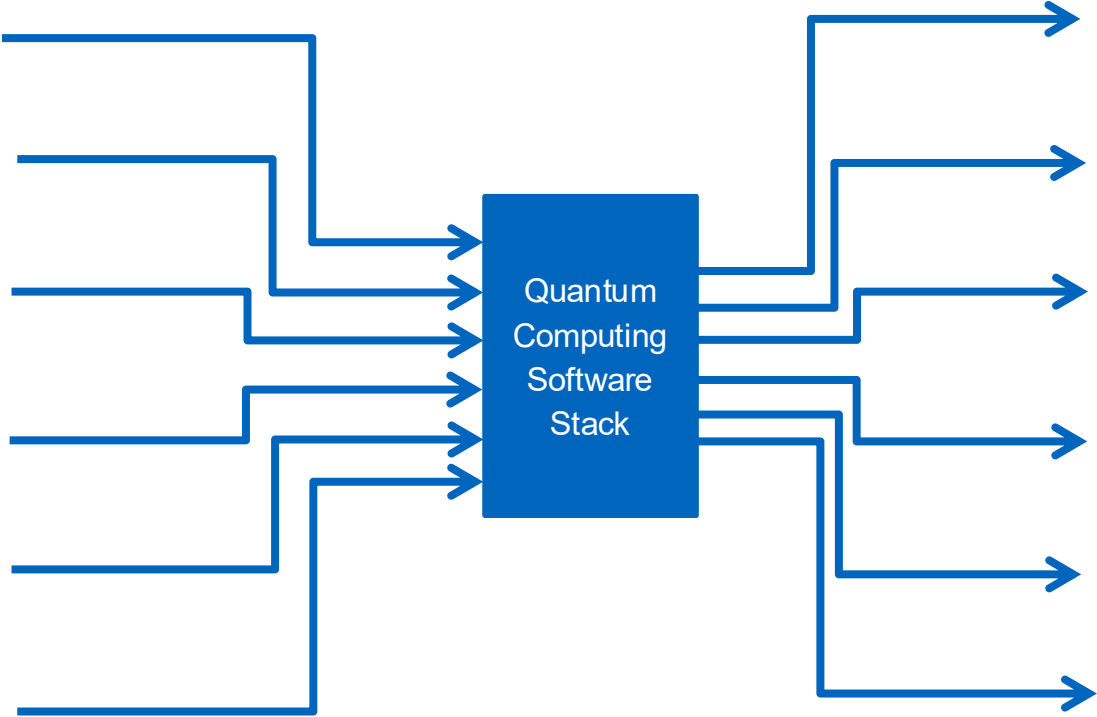


Quantum Devices

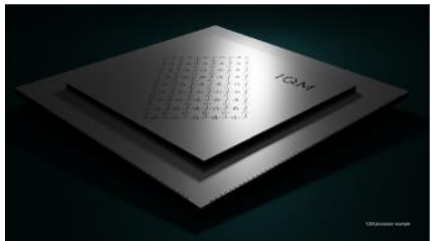


Quantum Computing – The Big Picture

Domain Experts



Quantum Devices



QC Software Stack Requirements

Support a wide range of QC modalities

- Find common abstraction among modalities
- Query properties from concrete systems where needed

Support a wide range of programming models and abstractions

- Decouple programming model from support stack, compiler and back-end
- Enable models from existing Python/Scripting models to high-performance approaches

Support HPCQC integration

- Integrate QC into the existing HPC ecosystem as a specialized accelerator
- Hybrid programming abstractions building on HPC models and languages

Support efficient resource utilization

- Efficient, hierarchical scheduling combined with resource and runtime predictions
- Optimize quantum compilation and optimization on modern hardware

Support easy system management

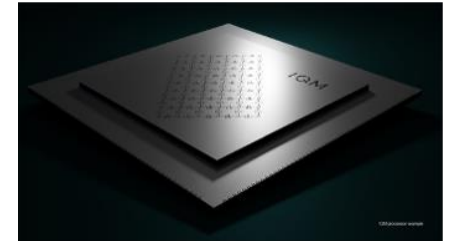
- Comprehensive monitoring and Online Data Analytics
- Easy user and system management through configurable, modern portal interface

Support internal and external users and installations

The Anatomy of the MQSS



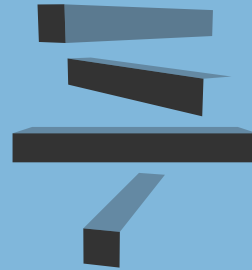
MQSS
Munich Quantum
Software Stack



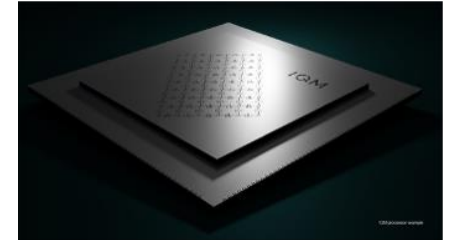
The Anatomy of the MQSS



Front-end Interface / Programming Models



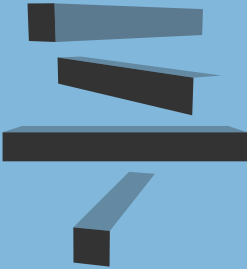
MQSS
Munich Quantum
Software Stack



The Anatomy of the MQSS

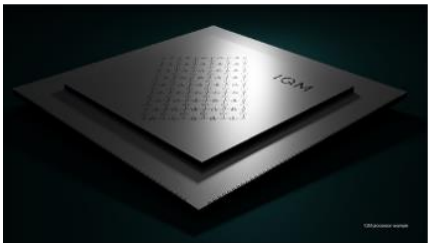


Front-end Interface / Programming Models



MQSS

Munich Quantum
Software Stack



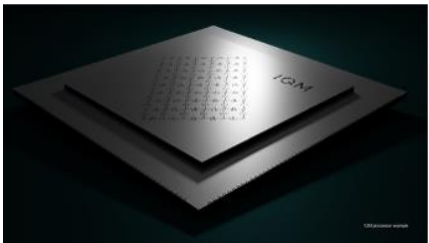
The Anatomy of the MQSS



Front-end Interface / Programming Models



Backend-end Interface



The Anatomy of the MQSS



The Anatomy of the MQSS



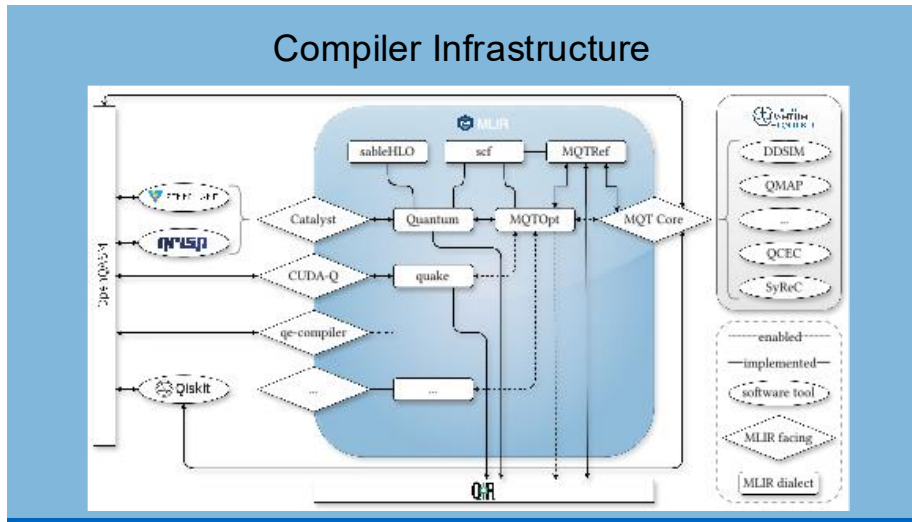
The Anatomy of the MQSS



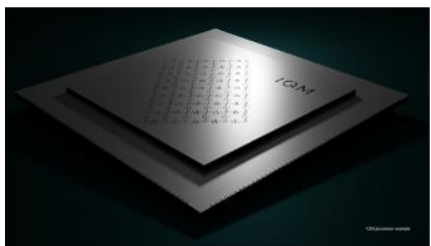
Front-end Interface / Programming Models

Resource Management / Scheduling

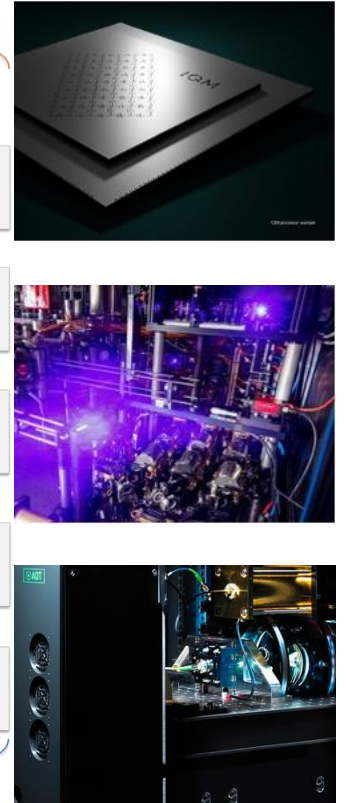
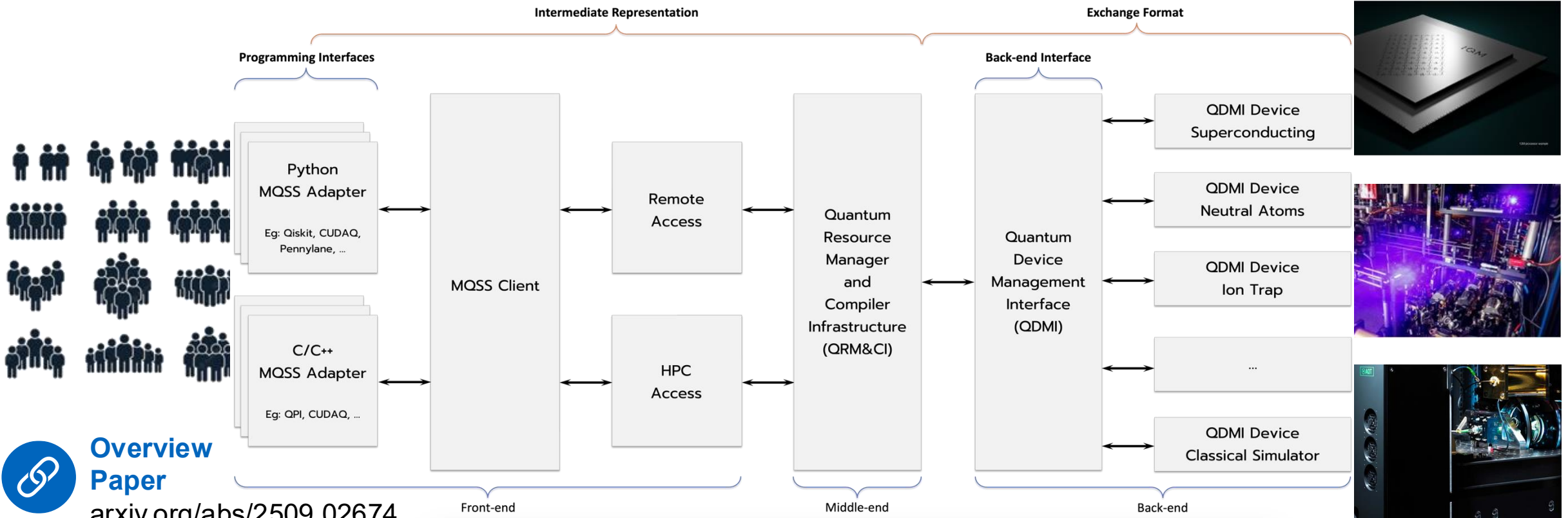




Back-end Interface



The Anatomy of the MQSS



Overview Paper

arxiv.org/abs/2509.02674



arXiv > quant-ph > arXiv:2509.02674

Quantum Physics

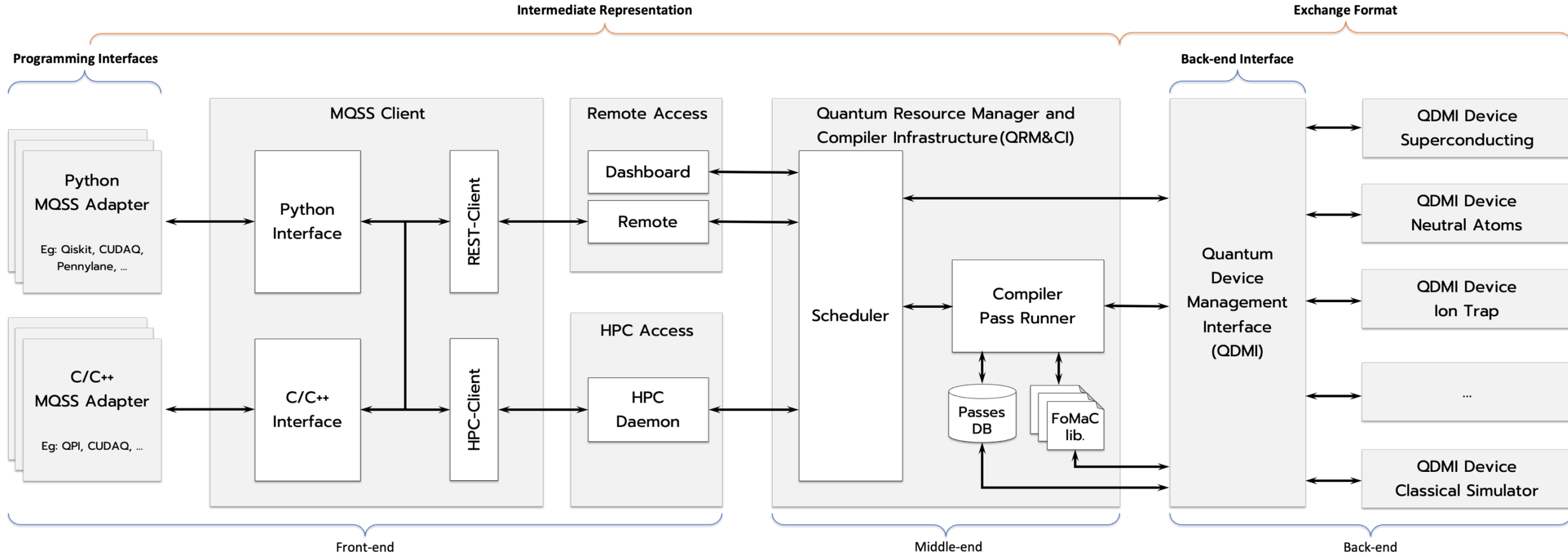
[Submitted on 2 Sep 2025]

The Munich Quantum Software Stack: Connecting End Users, Integrating Diverse Quantum Technologies, Accelerating HPC

Lukas Burgholzer, Jorge Echavarria, Patrick Hopf, Yannick Stade, Damian Rovara, Ludwig Schmid, Ercüment Kaya, Burak Mete, Muhammad Nufail Farooqi, Minh Chung, Marco De Pascale, Laura Schulz, Martin Schulz, Robert Wille

lrz

The Anatomy of the MQSS



arXiv > quant-ph > arXiv:2509.02674

Search... Help | Adv

Quantum Physics

[Submitted on 2 Sep 2025]

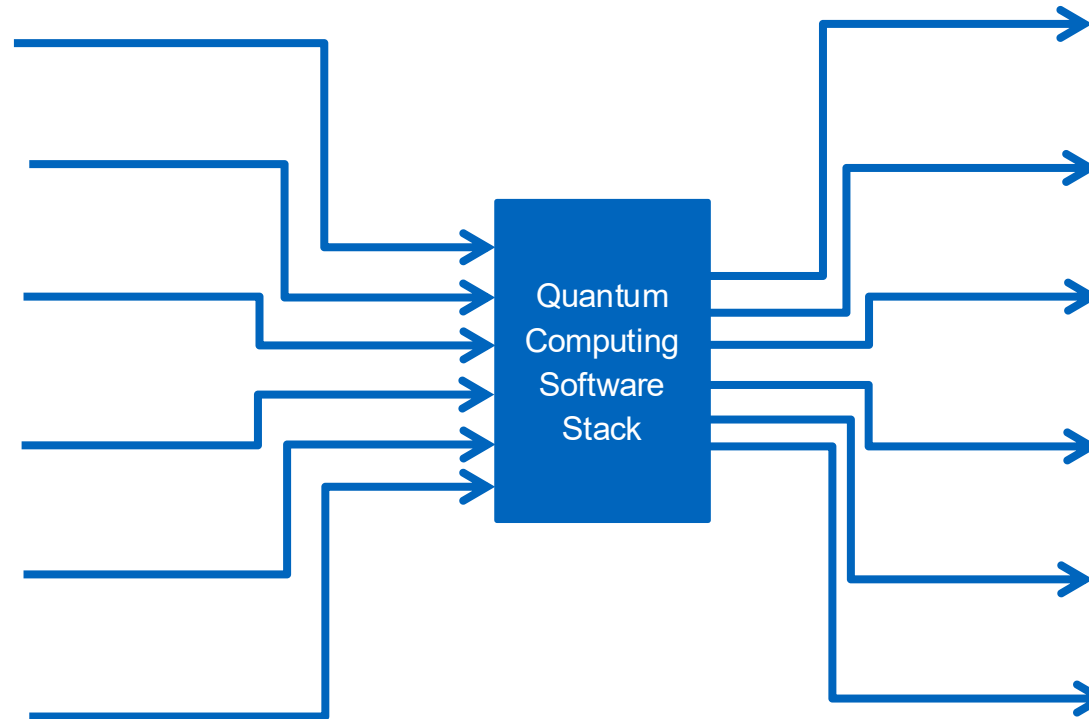
The Munich Quantum Software Stack: Connecting End Users, Integrating Diverse Quantum Technologies, Accelerating HPC

Lukas Burgholzer, Jorge Echavarria, Patrick Hopf, Yannick Stade, Damian Rovara, Ludwig Schmid, Ercüment Kaya, Burak Mete, Muhammad Nufail Farooqi, Minh Chung, Marco De Pascale, Laura Schulz, Martin Schulz, Robert Wille

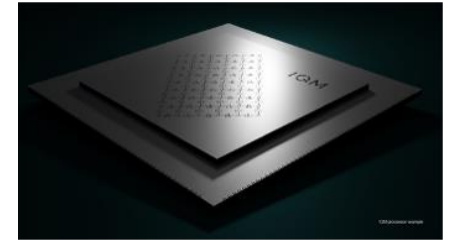
lrz

The Big Picture

Domain Experts

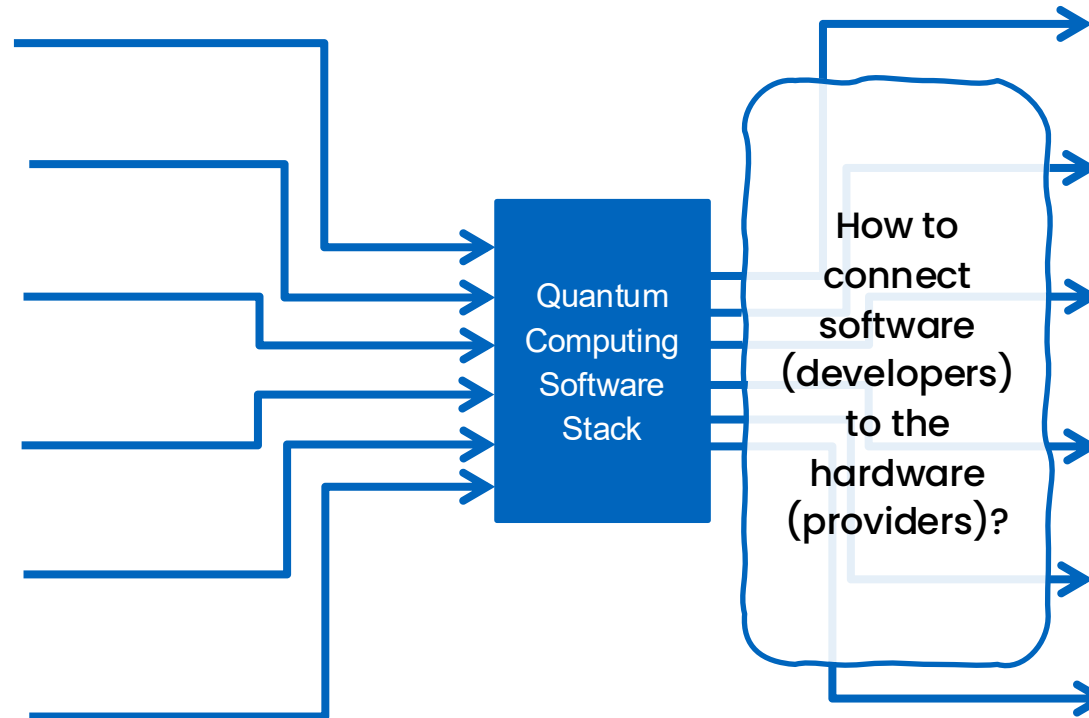


Quantum Devices

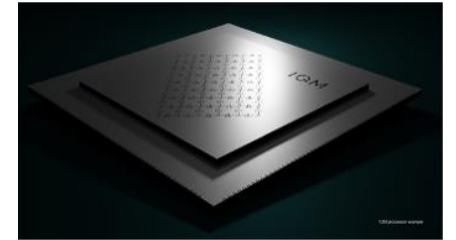


The Big Picture

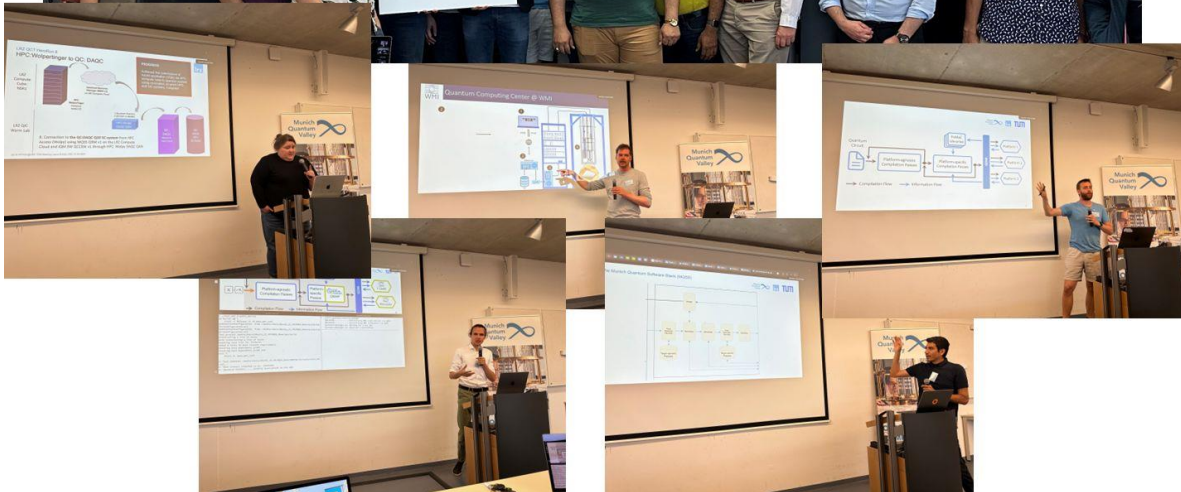
Domain Experts



Quantum Devices



Technical Exchange Meetings



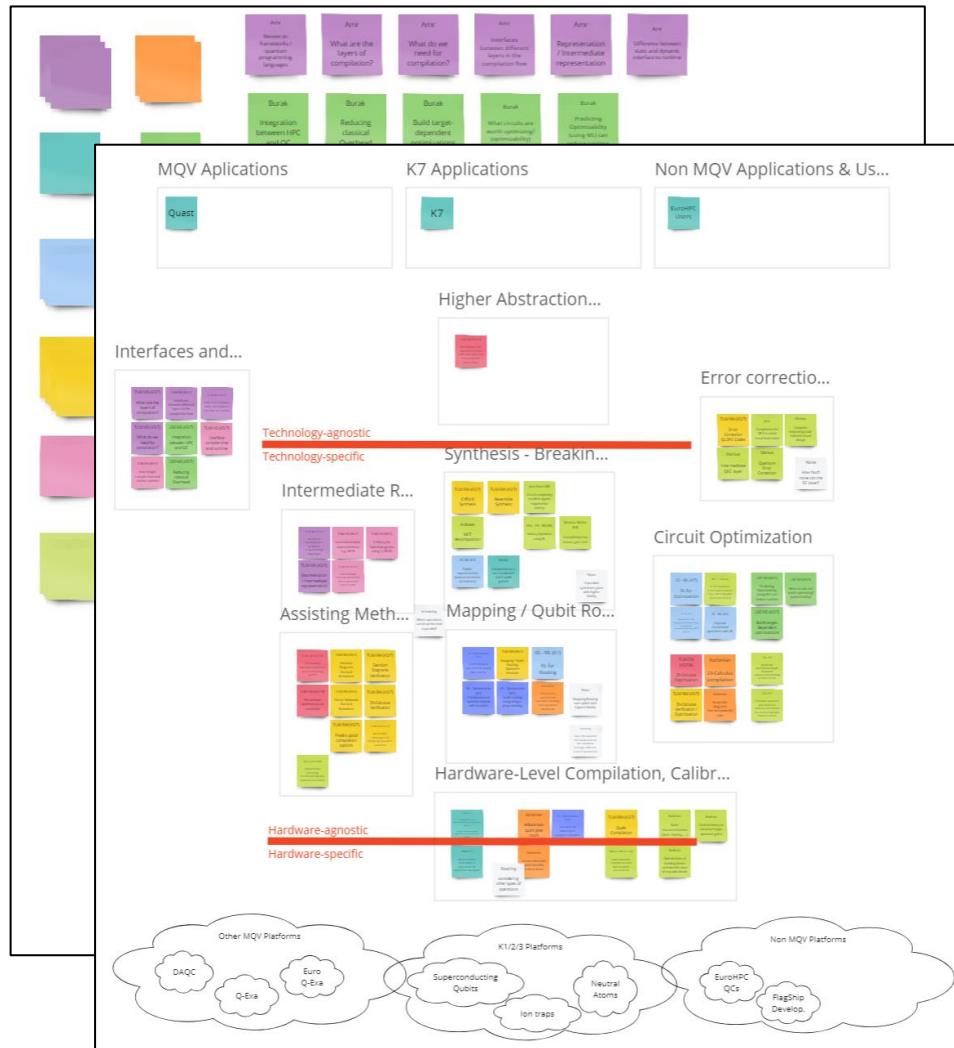
- Technical Exchange Meetings
- Origin/Motivation:
 - Triggered/started three years ago
 - Software as the “glue” that connects platforms with end users
 - Well, then let’s connect...
- Representatives from all “Ks” (consortia)
- Frequent meetings (approx. every 3-4 months)
- Triggered working groups
- Evolved to the **Munich Quantum Software Stack (MQSS)** initiative

■ Software Perspective



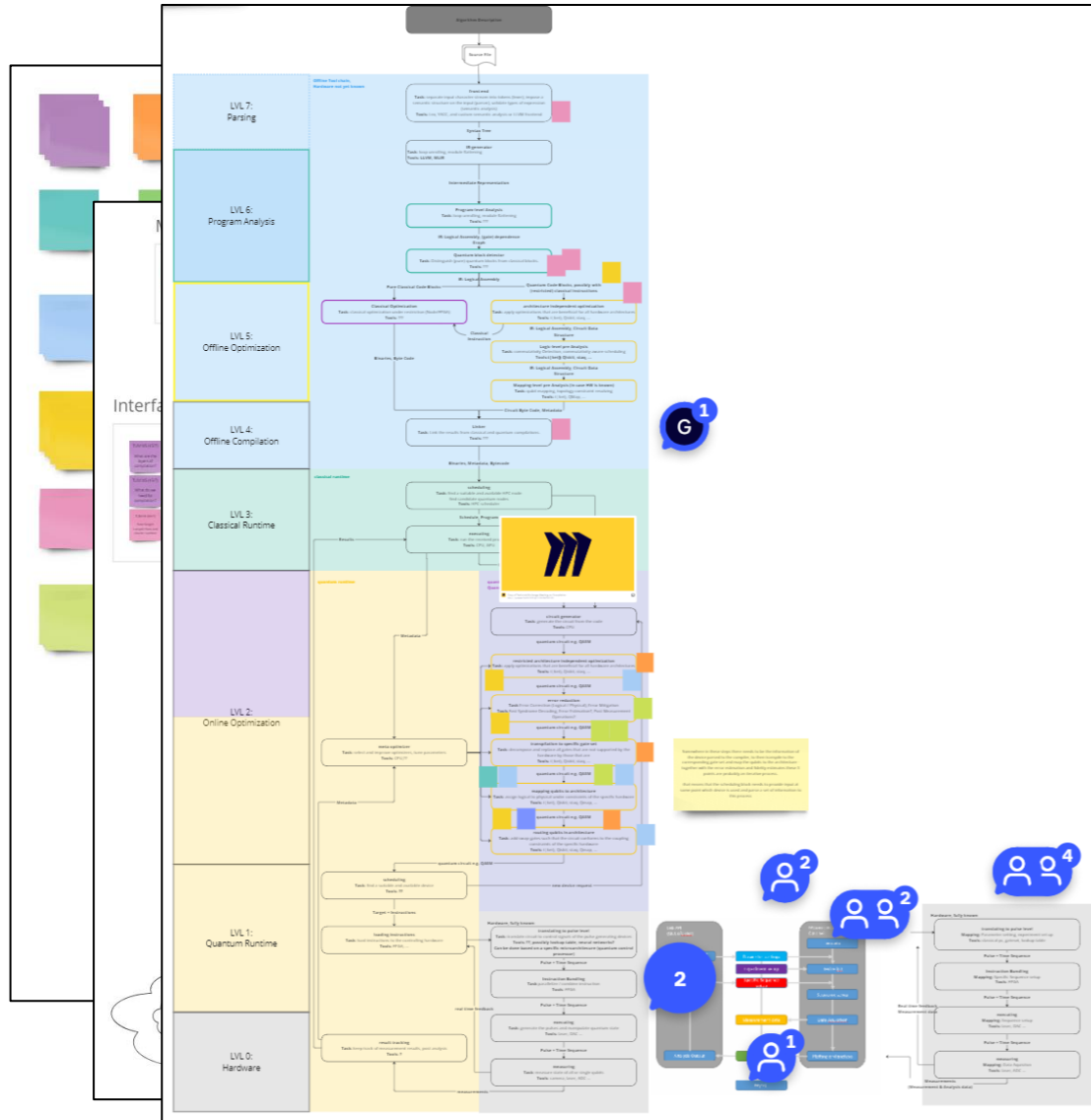
Technical Exchange Meetings

■ Software Perspective

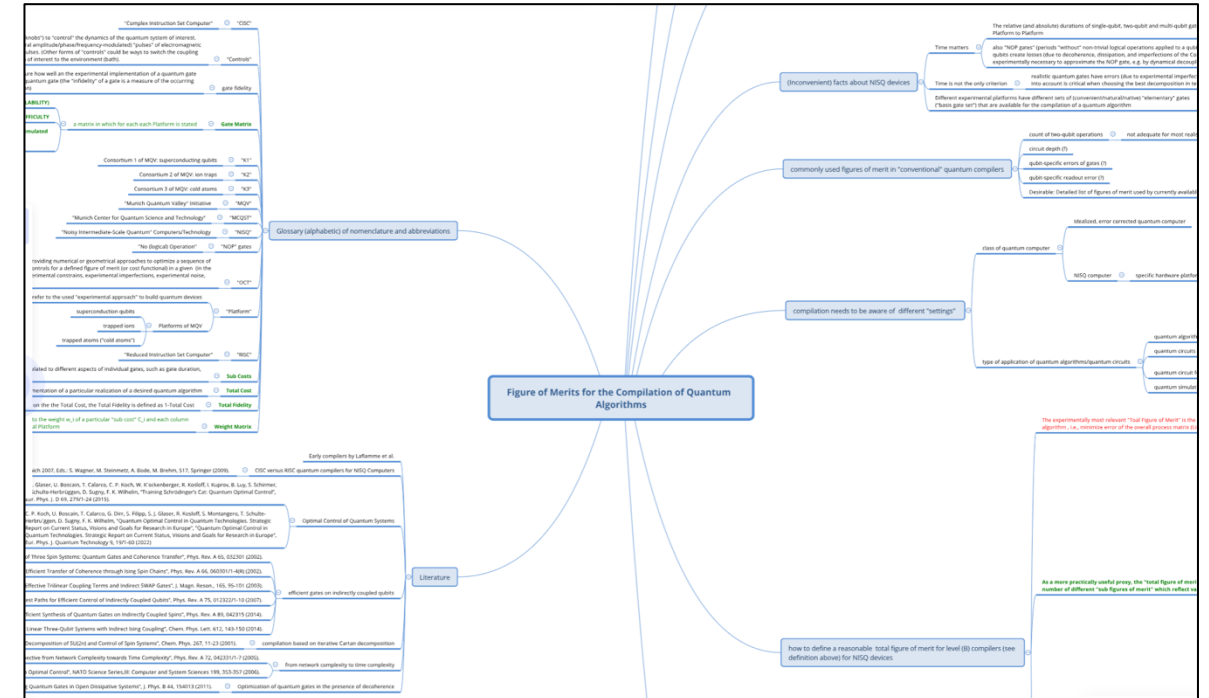


Technical Exchange Meetings

■ Software Perspective

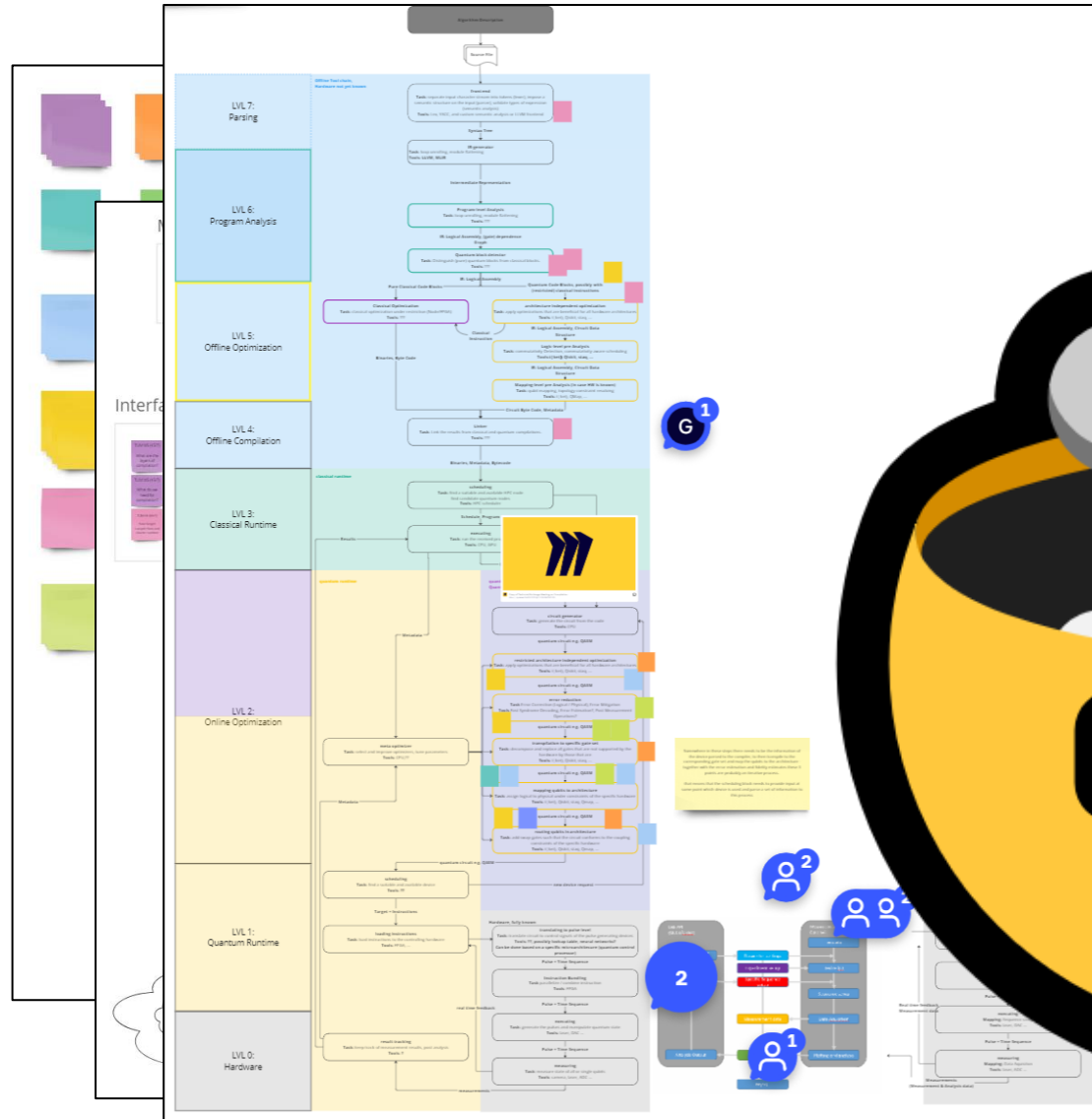


■ Experimentalist Perspective

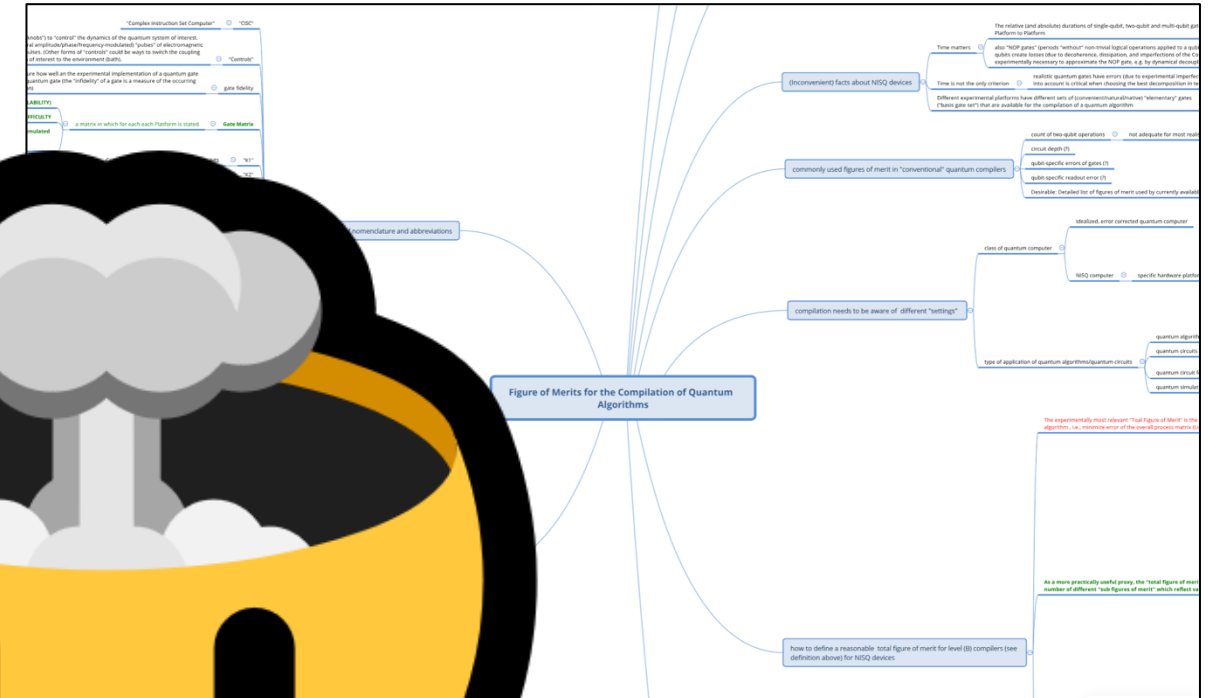


Technical Exchange Meetings

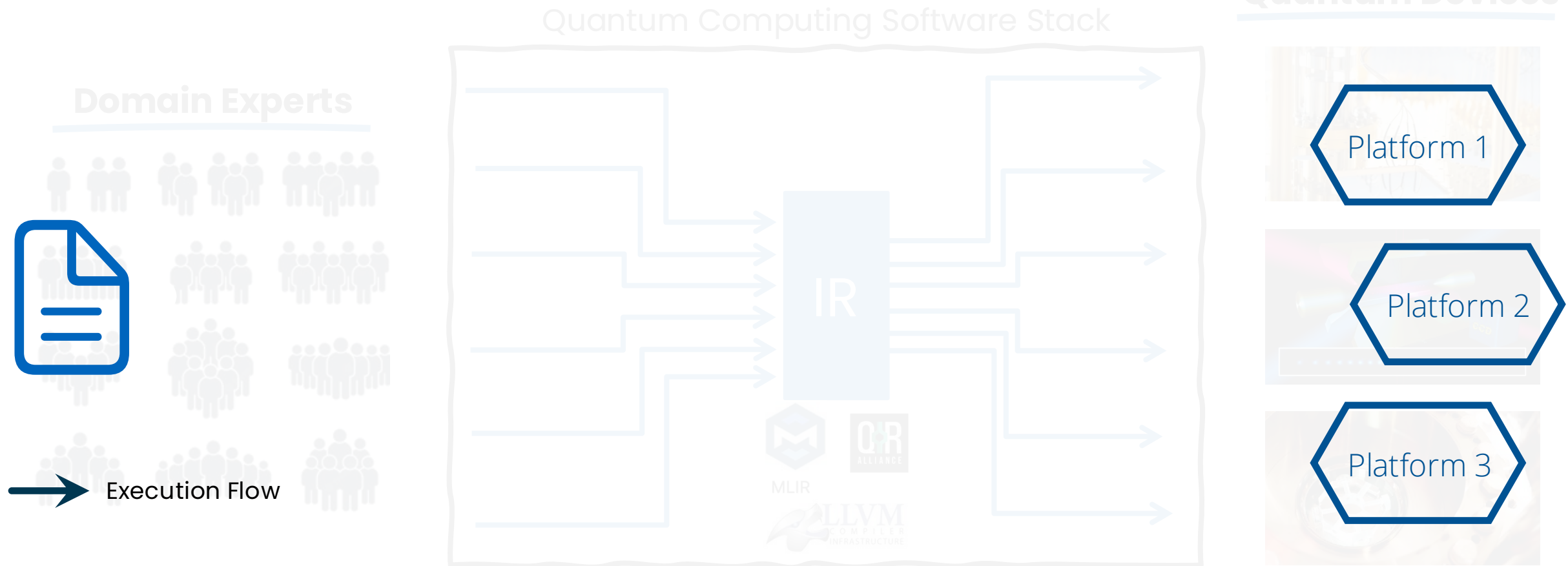
■ Software Perspective



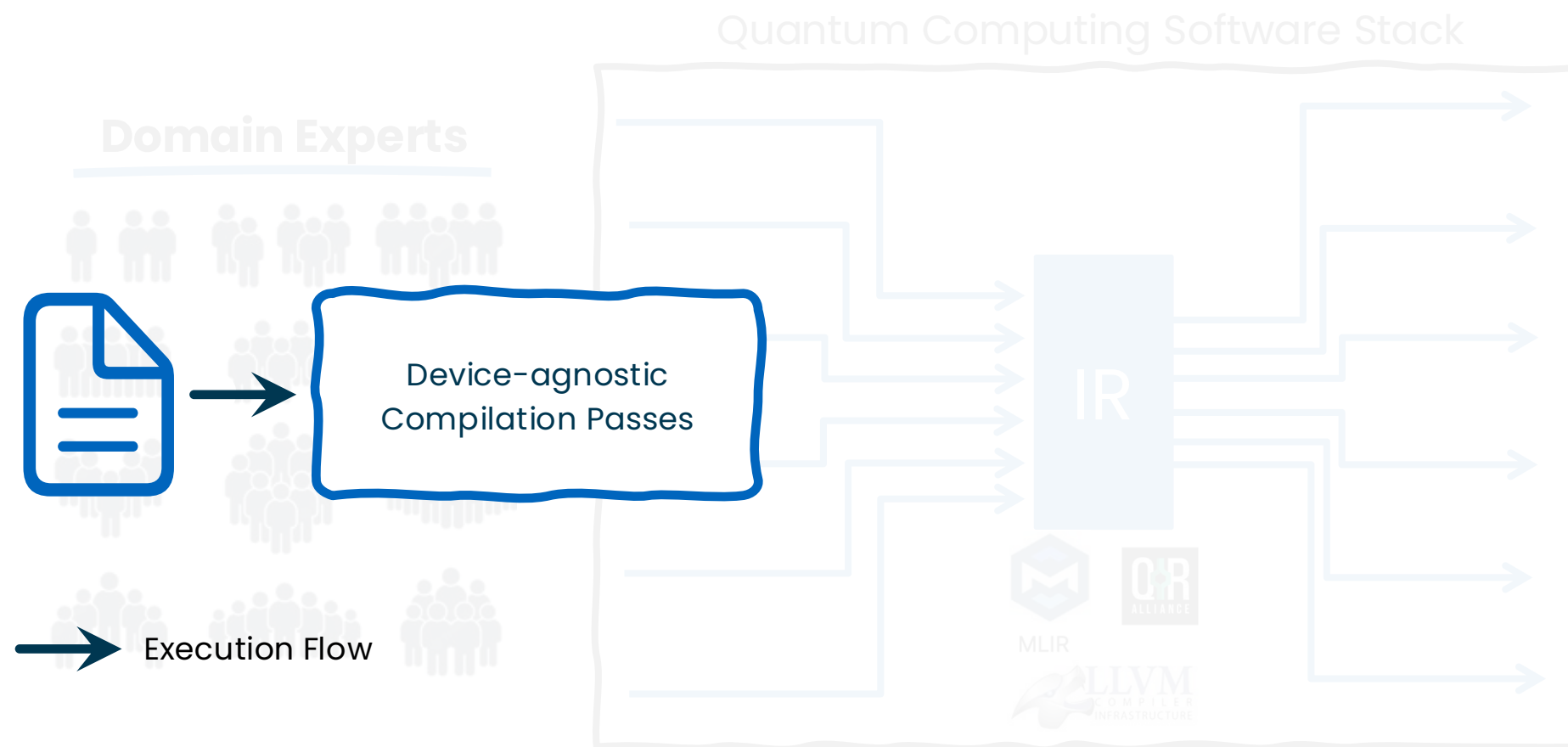
■ Experimentalist Perspective



QDMI The Quantum Device Management Interface



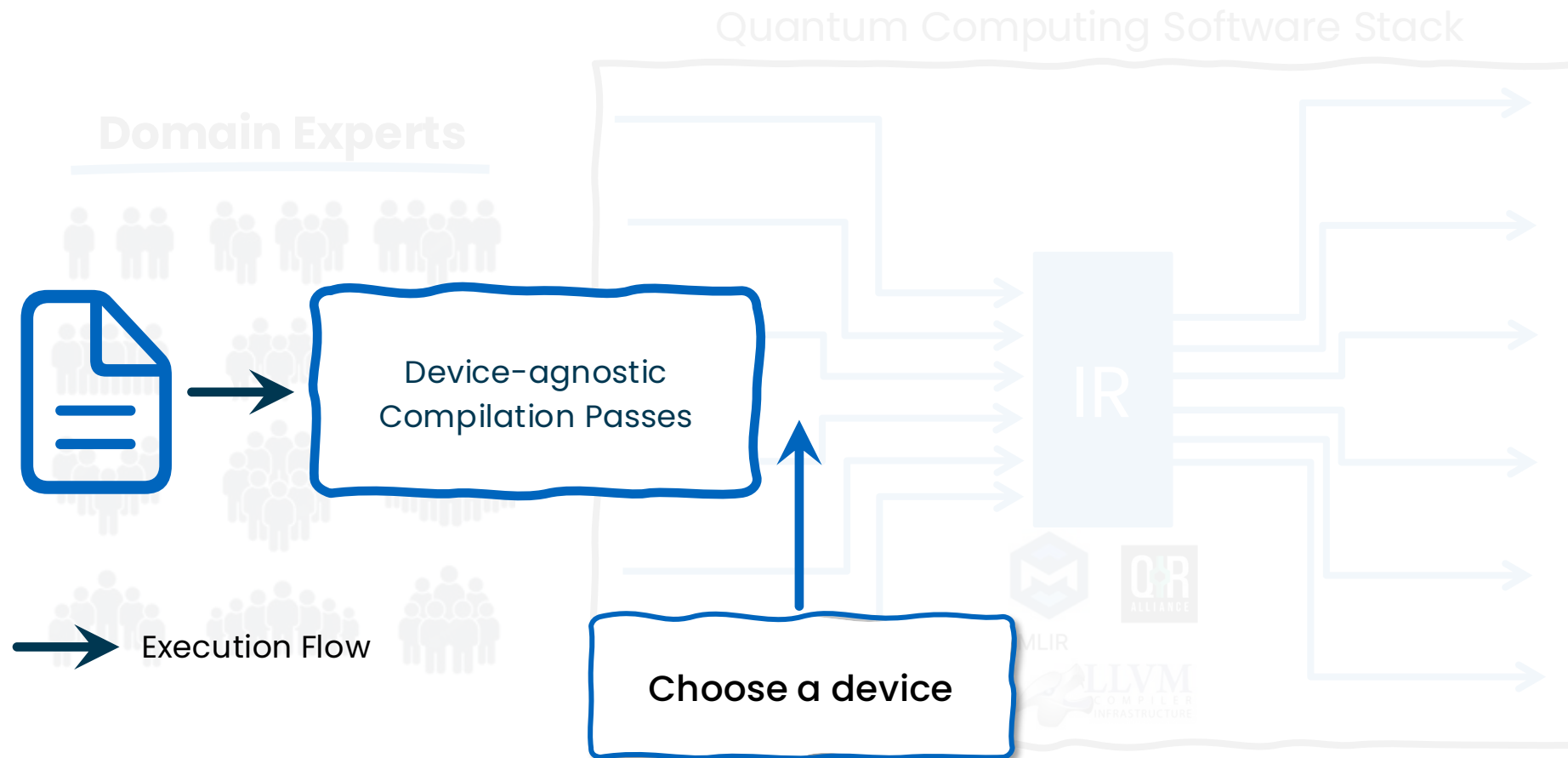
QDMI The Quantum Device Management Interface



Quantum Devices



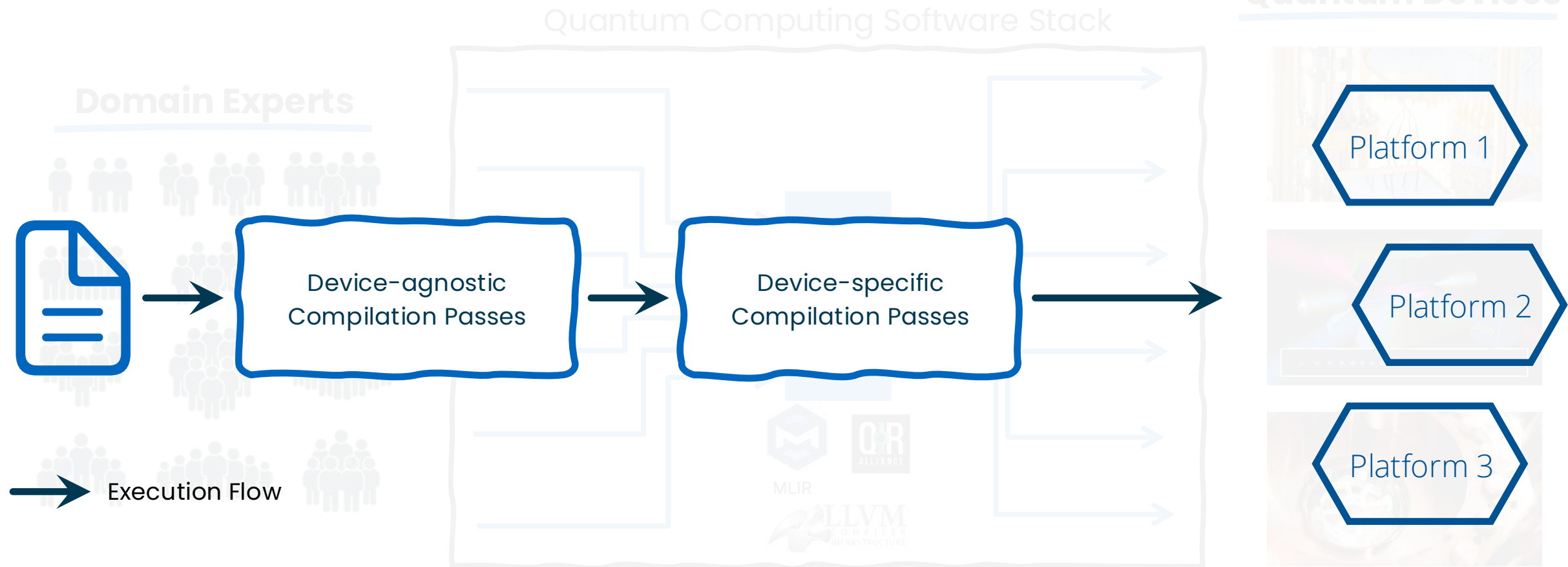
QDMI The Quantum Device Management Interface



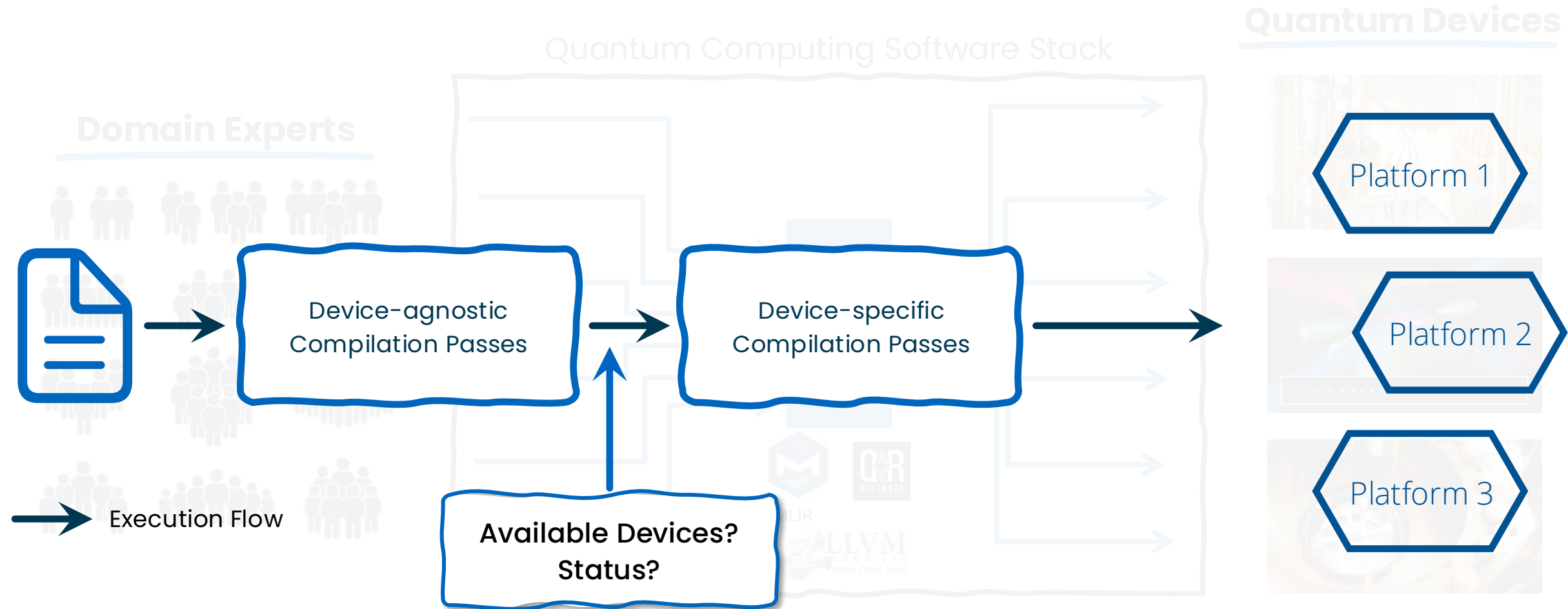
Quantum Devices



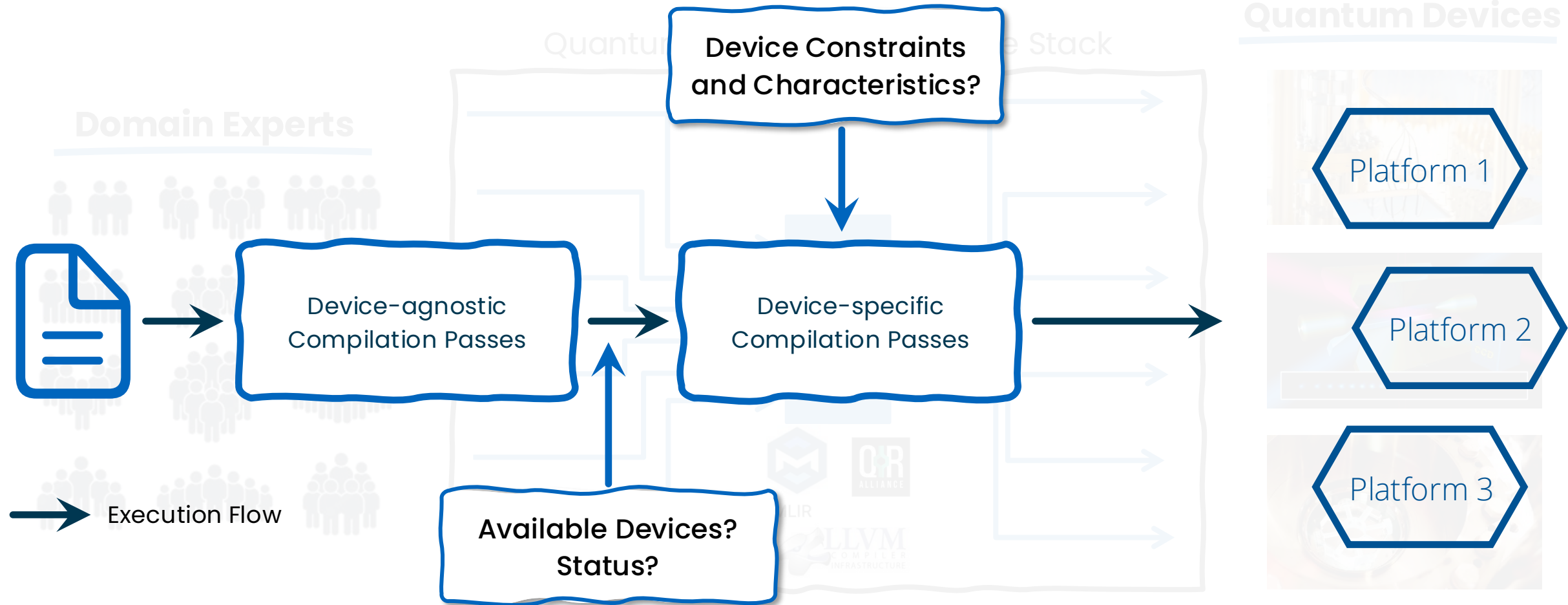
QDMI The Quantum Device Management Interface



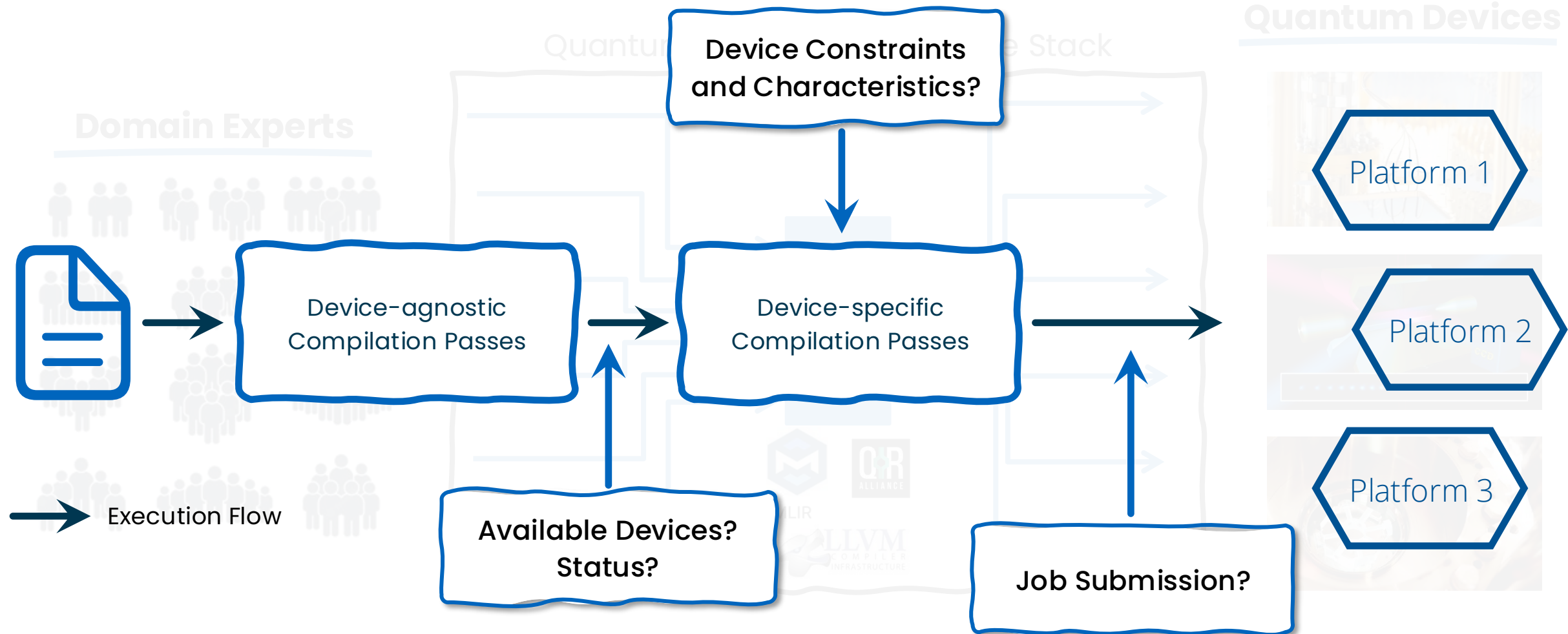
QDMI The Quantum Device Management Interface



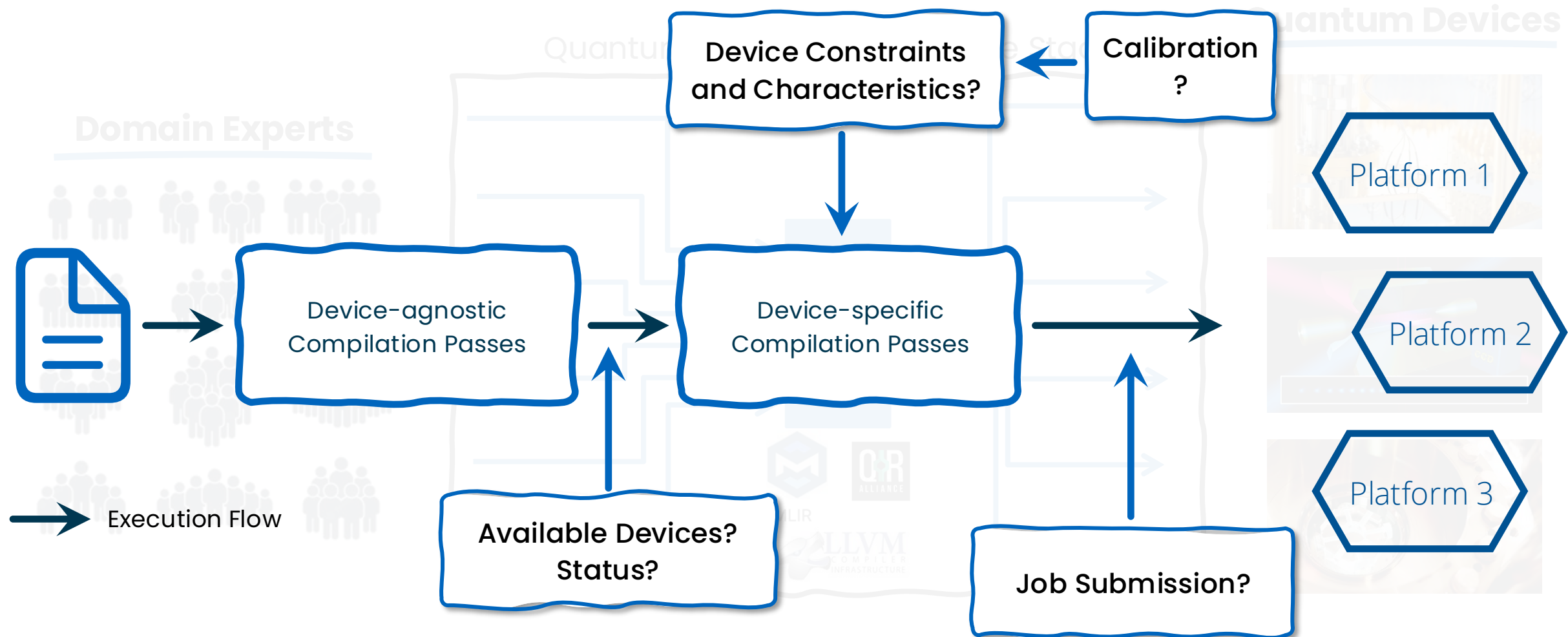
QDMI The Quantum Device Management Interface



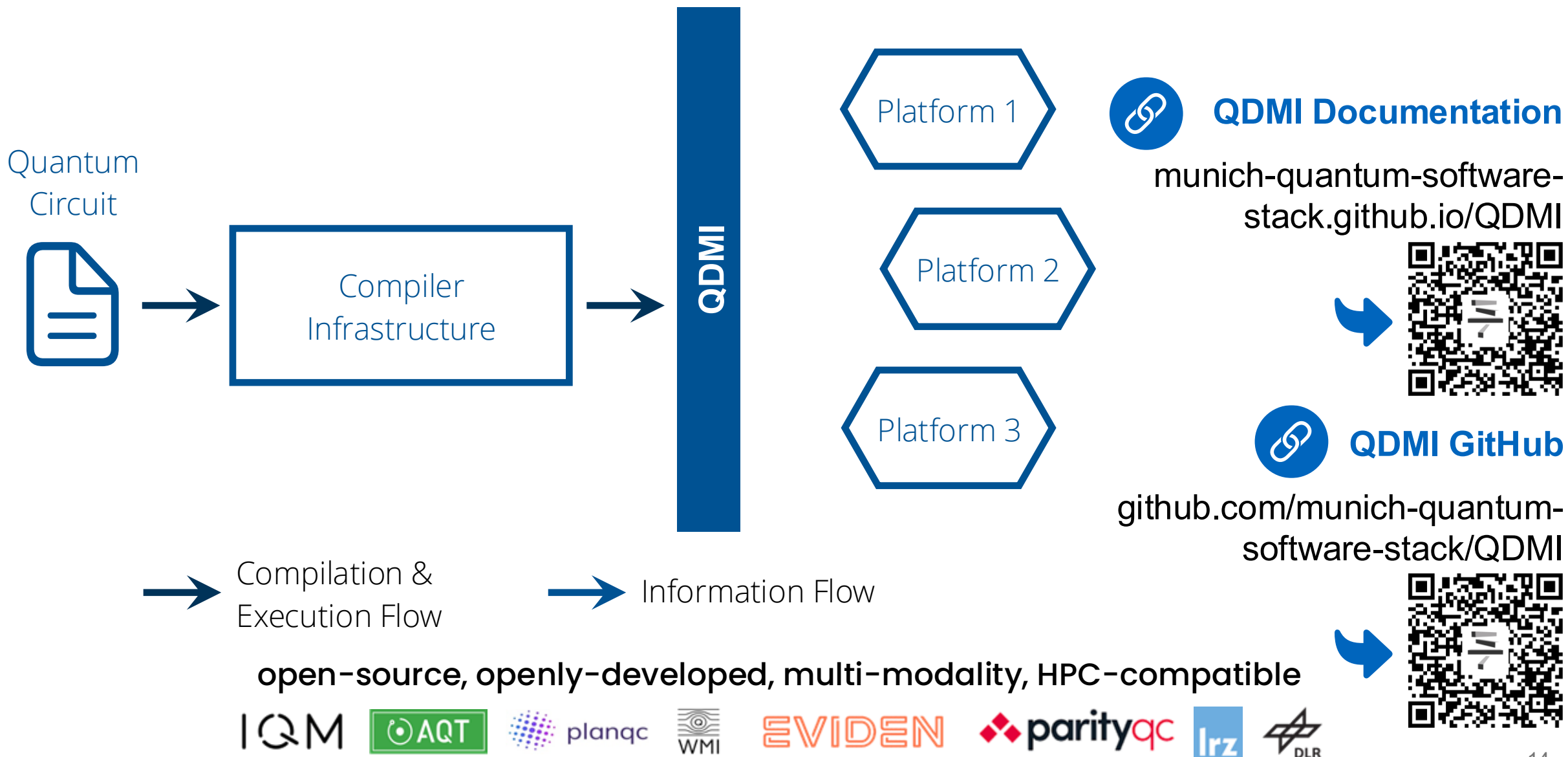
QDMI The Quantum Device Management Interface



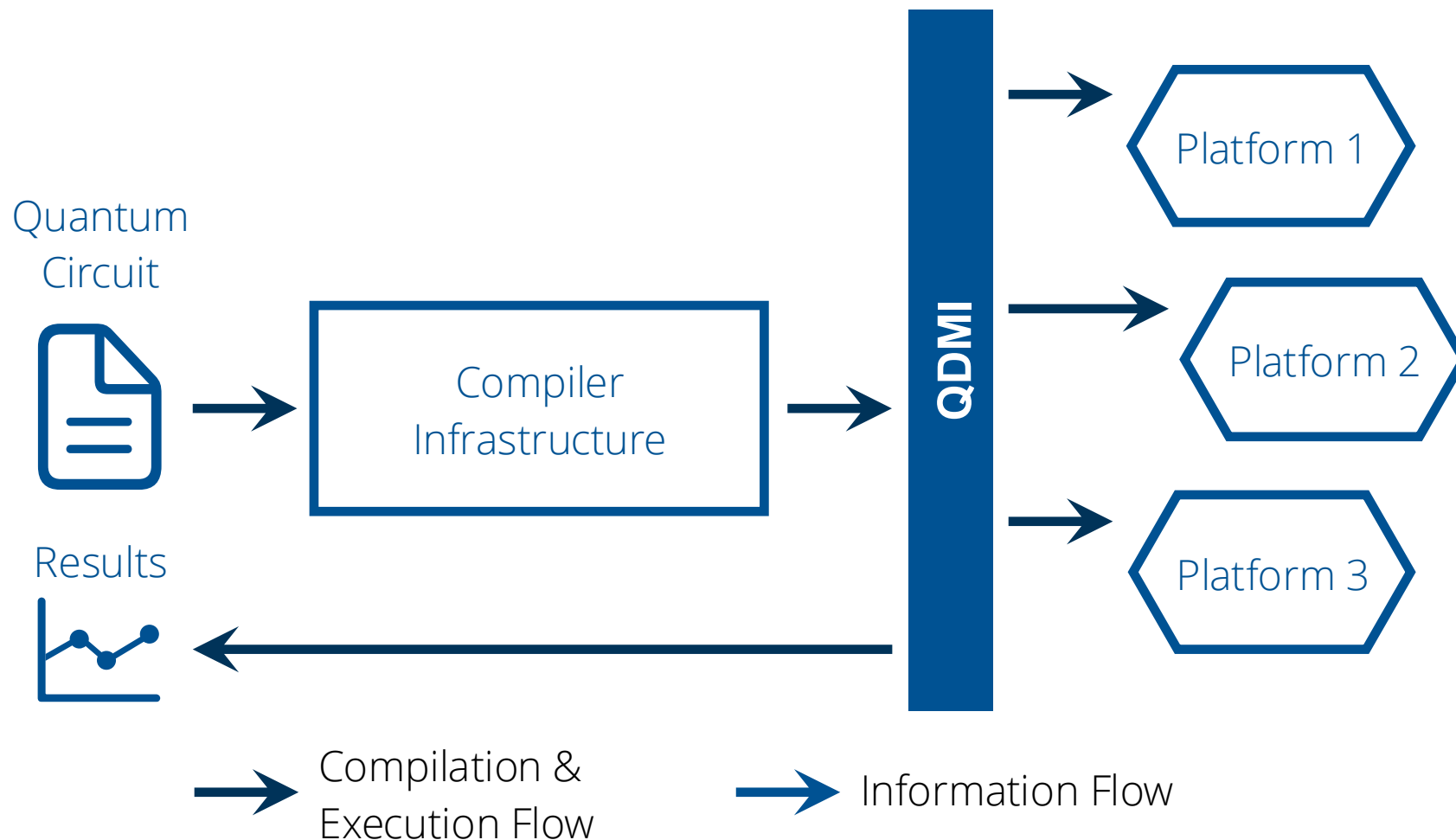
QDMI The Quantum Device Management Interface



QDMI The Quantum Device Management Interface



QDMI The Quantum Device Management Interface



QDMI Documentation

munich-quantum-software-stack.github.io/QDMI



QDMI GitHub

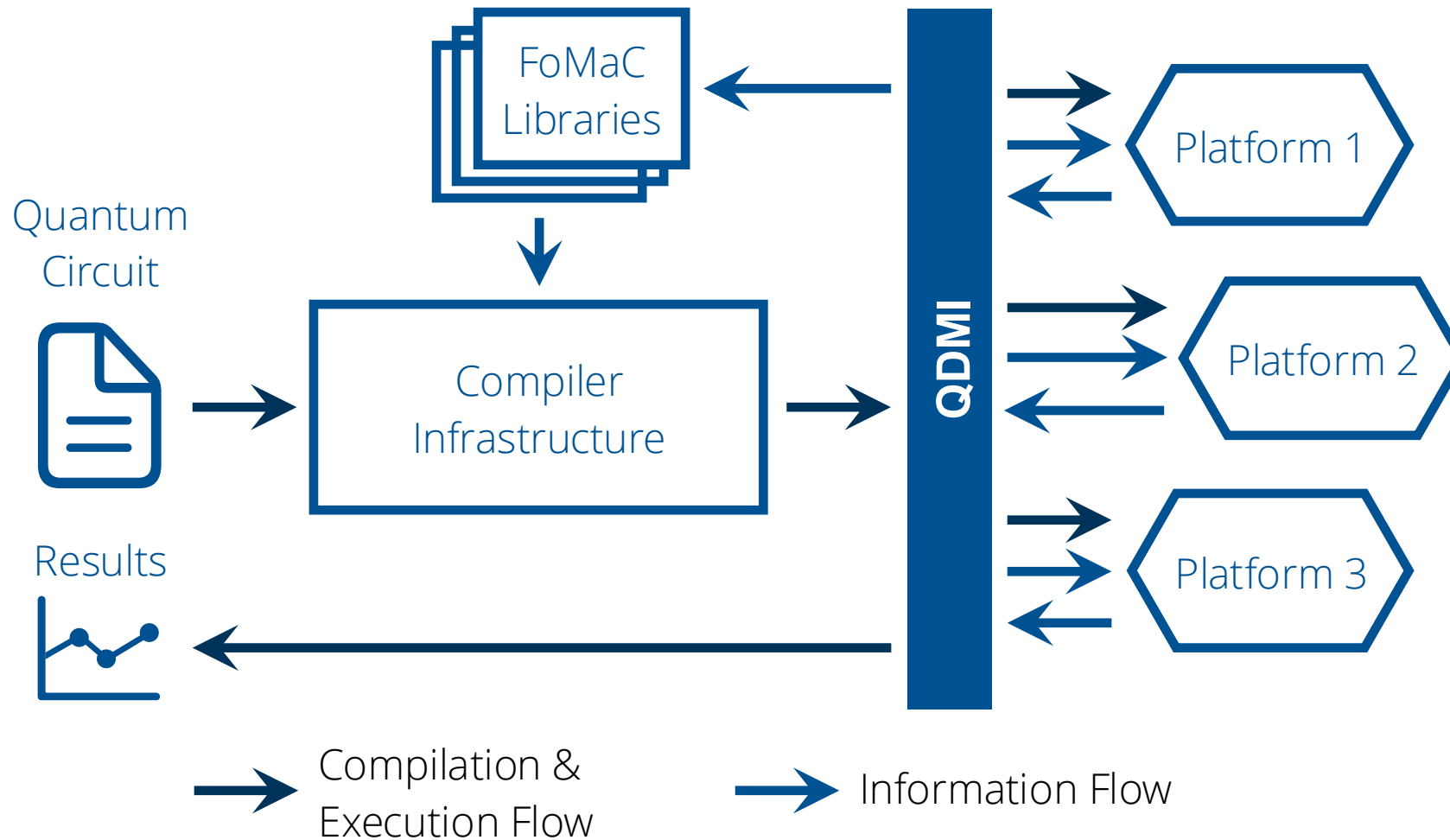
github.com/munich-quantum-software-stack/QDMI



open-source, openly-developed, multi-modality, HPC-compatible



QDMI The Quantum Device Management Interface



QDMI Documentation

munich-quantum-software-stack.github.io/QDMI



QDMI GitHub

github.com/munich-quantum-software-stack/QDMI



open-source, openly-developed, multi-modality, HPC-compatible

QDMI The Quantum Device Management Interface



Clients

Driver

Devices



QDMI The Quantum Device Management Interface



Session

- User Management
- Access Control
- Resource Management

Clients

Driver

Devices



QDMI The Quantum Device Management Interface



QDMI Device Session Interface

QDMI Device Interface

Description

Provides functions to manage sessions between the driver and device.

A device session is a connection between a driver and a device that allows the driver to interact with the device. Sessions are used to authenticate with the device and to manage resources required for the interaction with the device.

The typical workflow for a device session is as follows:

- Allocate a session with `QDMI_device_session_alloc`.
- Set parameters for the session with `QDMI_device_session_set_parameter`.
- Initialize the session with `QDMI_device_session_init`.
- Run code to interact with the device using the `device query interface` and the `device job interface`.
- Free the session with `QDMI_device_session_free` when it is no longer needed.



QDMI The Quantum Device Management Interface



Session

- User Management
- Access Control
- Resource Management

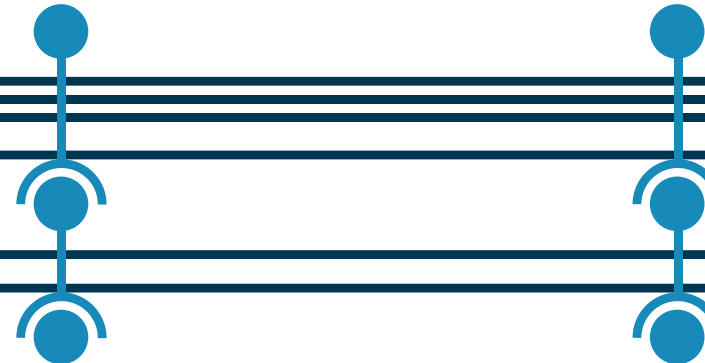
Query

- Device Properties
- Site Properties
- Operation Properties

Clients

Driver

Devices



QDMI The Quantum Device Management Interface



Clients

Driver

Devices

QDMI Device Query Interface

QDMI Device Interface

Description

Provides functions to query properties of a device.

The query interface enables to query static and dynamic properties of a device and its constituents in a unified fashion. It operates on [QDMI_Device_Session](#) handles created via the [device session interface](#).

Functions

```
int QDMI_device_session_query_device_property (QDMI_Device_Session session,
QDMI_Device_Property prop, size_t size, void *value, size_t *size_ret)
```

Query a device property.

```
int QDMI_device_session_query_site_property (QDMI_Device_Session session, QDMI_Site
site, QDMI_Site_Property prop, size_t size, void *value, size_t *size_ret)
```

Query a site property.

```
int QDMI_device_session_query_operation_property (QDMI_Device_Session session,
QDMI_Operation operation, size_t num_sites, const QDMI_Site *sites, size_t
num_params, const double *params, QDMI_Operation_Property prop, size_t size,
void *value, size_t *size_ret)
```

Query an operation property.



QDMI The Quantum Device Management Interface



Session

- User Management
- Access Control
- Resource Management

Query

- Device Properties
- Site Properties
- Operation Properties

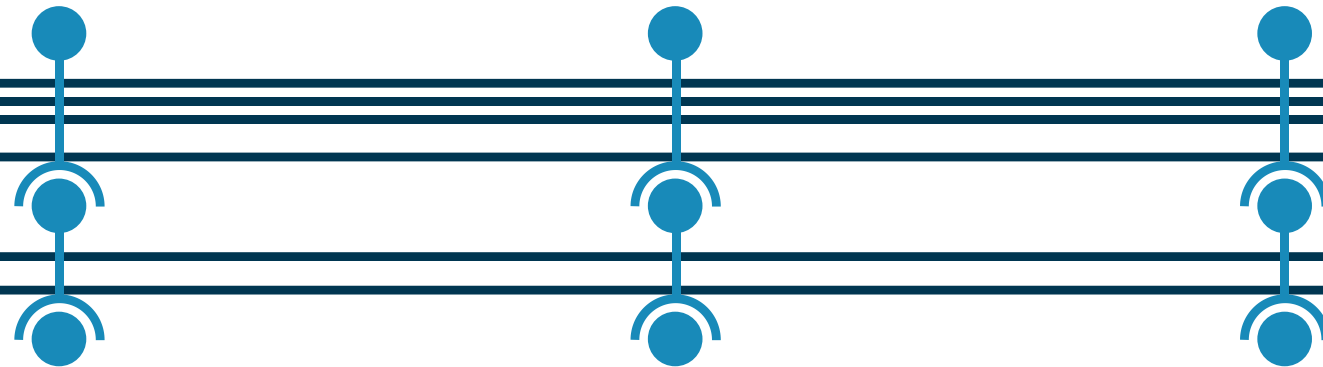
Job

- Job Configuration
- Job Submission
- Result Retrieval

Clients

Driver

Devices



QDMI The Quantum Device Management Interface



Session

- User Management
- Access Control
- Resource Management

Clients

Driver

Devices

QDMI Device Job Interface

QDMI Device Interface

Description

Provides functions to manage jobs on a device.

A job is a task submitted to a device for execution. Most jobs are quantum circuits to be executed on a quantum device. However, jobs can also be a different type of task, such as calibration.

The typical workflow for a device job is as follows:

- Create a job with `QDMI_device_session_create_device_job`.
- Set parameters for the job with `QDMI_device_job_set_parameter`.
- Submit the job with `QDMI_device_job_submit`.
- Check the status of the job with `QDMI_device_job_check`.
- Wait for the job to finish with `QDMI_device_job_wait`.
- Retrieve the results of the job with `QDMI_device_job_get_results`.
- Free the job with `QDMI_device_job_free` when it is no longer used.



QDMI The Quantum Device Management Interface

QDMI Client Query Interface

Description

Provides functions to query properties of devices.

The query interface enables to query static and dynamic properties of devices and their constituents in a unified fashion. It operates on `QDMI_Device` handles queried from a `QDMI_Session` via `QDMI_session_query_session_property`.

Functions

```
int QDMI_device_query_device_property (QDMI_Device device, QDMI_Device_Property prop, size_t size, void *value, size_t *size_ret)
    Query a device property.

int QDMI_device_query_site_property (QDMI_Device device, QDMI_Site site, QDMI_Site_Property prop, size_t size, void *value, size_t *size_ret)
    Query a site property.

int QDMI_device_query_operation_property (QDMI_Device device, QDMI_Operation operation, size_t num_sites, const QDMI_Site *sites, size_t num_params, const double *params, QDMI_Operation_Property prop, size_t size, void *value, size_t *size_ret)
    Query an operation property.
```

Full API Documentation

Driver

Devices

Session

Management

S Cont

orce

Query

• Device Properties

• Site Properties

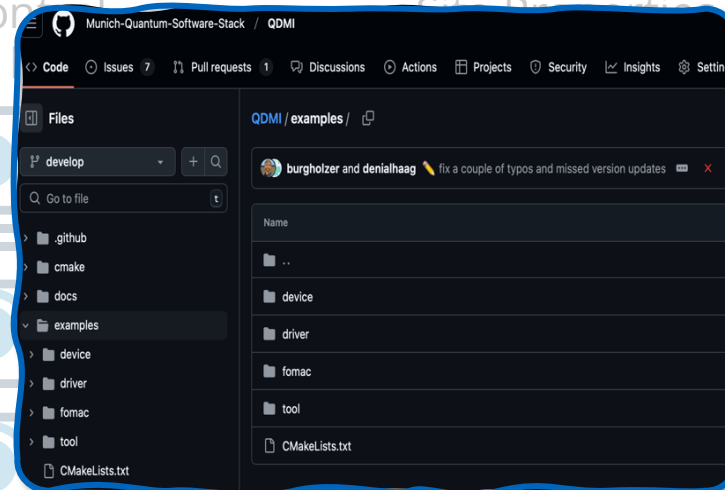
• Operation Properties

Job

• Job Configuration

• Job Submission

• Result Retrieval



Reference Implementations

Development Guide

Ready to contribute to QDMI? This guide will help you get started.

Initial Setup

1. Fork the `QDMI` repository on GitHub (see <https://docs.github.com/en/get-started/quickstart/fork-a-repo>).
2. Clone your fork locally

```
git clone git@github.com:your_name_here/QDMI.git
```

3. Change into the project directory

```
cd QDMI
```

4. Create a branch for local development

```
git checkout -b name-of-your-bugfix-or-feature
```

Now you can make your changes locally.

5. (Optional, **highly recommended**) Install `pre-commit` to automatically run a set of checks before each commit.

```
via uv   via brew   via pipx   via pip
```

The easiest way to install `pre-commit` is via `uv`.

```
uv tool install pre-commit
```

Development Guide

Extending QDMI for Neutral Atoms



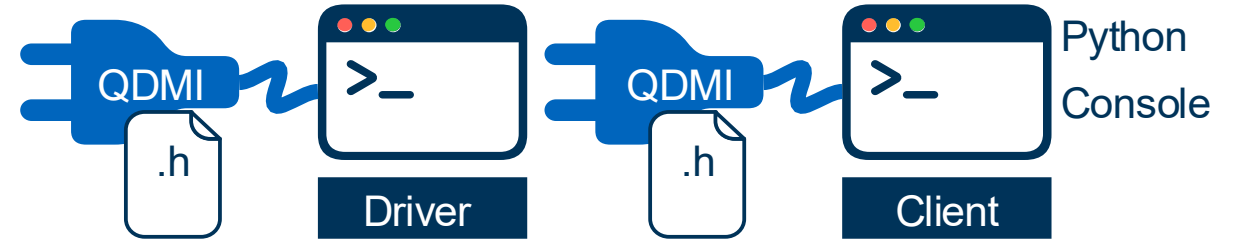
```

Python
>>> from mqt.core.fomac import *
>>> device = devices()[0]
>>> device.name()
'MQT NA Default QDMI Device'
>>> site = device.sites()[24]
>>> site.x_coordinate(), site.y_coordinate()
(1, 2)
>>> [op.name() for op in device.operations()]
['ry', 'cz', 'rz', 'cz', 'load<0>', 'move<0>', 'store<0>']

```



Extending QDMI for Neutral Atoms



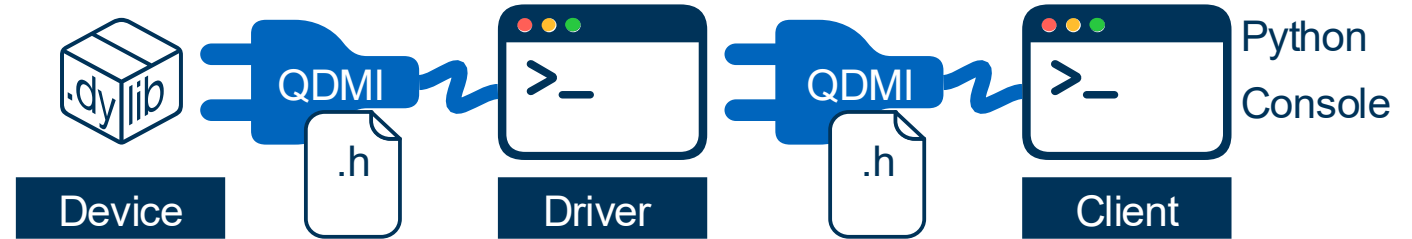
```

Python
>>> from mqt.core.fomac import *
>>> device = devices()[0]
>>> device.name()
'MQT NA Default QDMI Device'
>>> site = device.sites()[24]
>>> site.x_coordinate(), site.y_coordinate()
(1, 2)
>>> [op.name() for op in device.operations()]
['ry', 'cz', 'rz', 'cz', 'load<0>', 'move<0>', 'store<0>']

```



Extending QDMI for Neutral Atoms



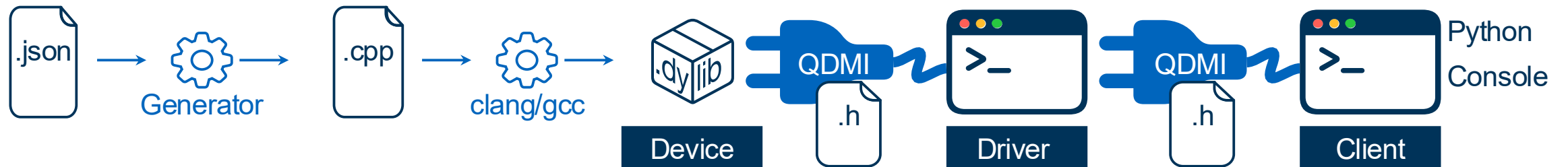
```
int MQT_NA_QDMI_device_session_query_device_property(
    MQT_NA_QDMI_Device_Session session,
    QDMI_Device_Property prop,
    size_t size, void* value, size_t* size_ret);

int MQT_NA_QDMI_device_session_query_site_property(
    MQT_NA_QDMI_Device_Session session,
    MQT_NA_QDMI_Site site, QDMI_Site_Property prop, ...);

int MQT_NA_QDMI_device_session_query_operation_property(
    MQT_NA_QDMI_Device_Session session,
    MQT_NA_QDMI_Operation operation,
    size_t num_sites, const MQT_NA_QDMI_Site* sites,
    size_t num_params, const double* params,
    QDMI_Operation_Property prop, ...);
```



Extending QDMI for Neutral Atoms



MQT Core

106 ★

638k



```

Shell
$ (uv) pip install mqt.core
  
```



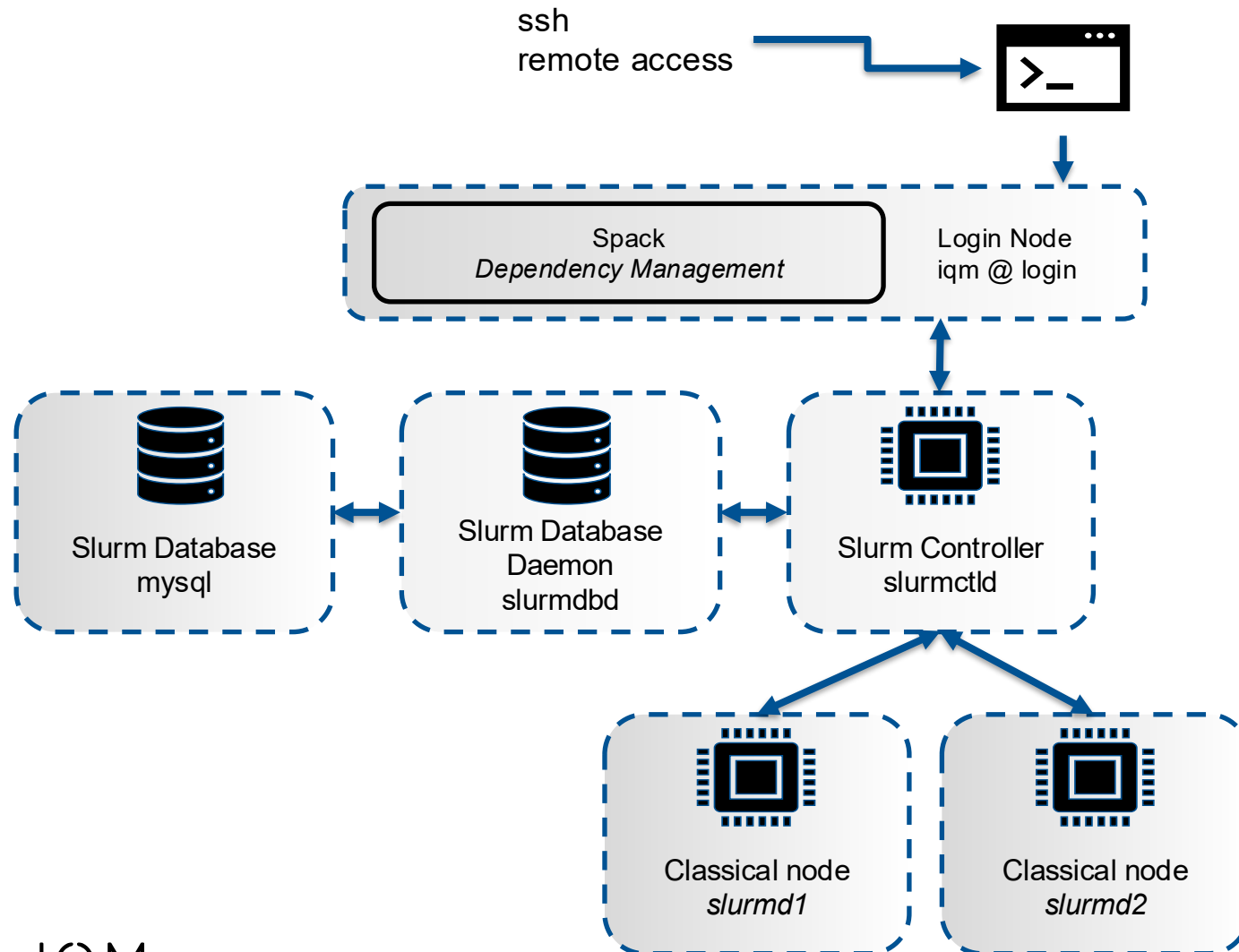
MQT Core (incl. QDMI Device Implementation)
github.com/munich-quantum-toolkit/core



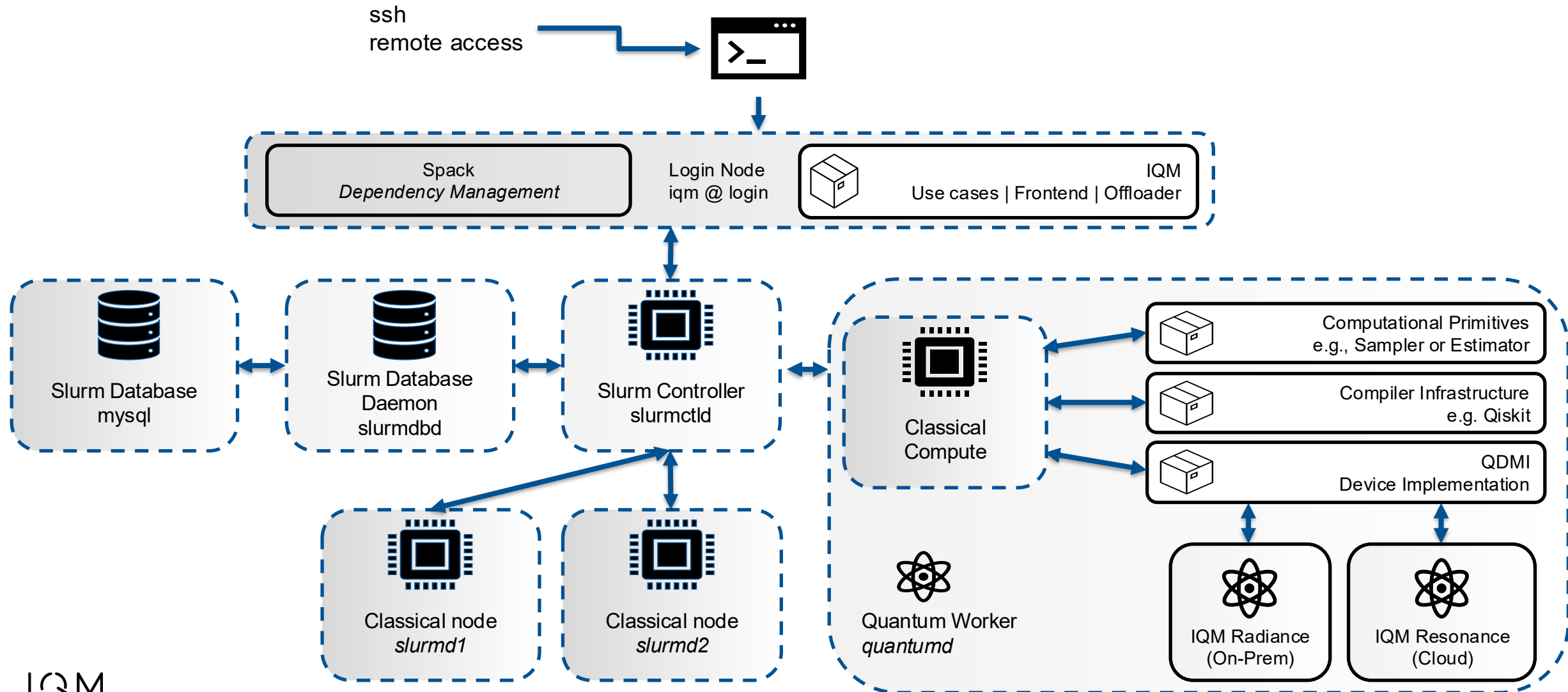
MQT Core Documentation
mqt.readthedocs.io/projects/core/en/latest/



How QDMI ties into the HPCQC Infrastructure



How QDMI ties into the HPCQC Infrastructure



Running QSCI

```
iqm @ login

>>> from iqm.aurora import run_qsci
>>> run_qsci(
    atom="H 0 0 0; H 0 0 1;", basis="sto-3g",
    offload=True, simulator=False)
'QSCI result = -1.101150330232619'
```



Running QSCI

iqm @ login

```
>>> from iqm.aurora import run_qsci
>>> run_qsci(
    atom="H 0 0 0; H 0 0 1;", basis="sto-3g",
    offload=True, simulator=False)
'QSCI result = -1.101150330232619'
```

qsci @ login

```
>>> driver = PySCFDriver(atom=atom, basis=basis)
    problem = driver.run()
    nuclear_repulsion_energy = problem.nuclear_repulsion_energy

>>> ansatz_circ = prepare_ansatz(
    problem.num_spatial_orbitals, problem.num_particles, problem.second_q_ops()[0],
    offload=offload, simulator=simulator)

>>> counts = sample_slurm(
    ansatz_circ, shots=10000, simulator=simulator)

>>> eigval = postprocess(atom, basis, counts)
>>> print(f"QSCI result = {eigval + nuclear_repulsion_energy:.15f}")
```


Running QSCI

offloader @ login

```
>>> qpy.dump(ansatz, "/jobs/ansatz.qpy")

>>> process = subprocess.run(
    srun --job-name="sample" --partition=quantum \
    sampler "/jobs/ansatz.qpy" str(shots)
)

>>> counts = eval(process.stdout)
```

sampler @ quantumd

```
>>> qc = qpy.load(args.circuit)[0]

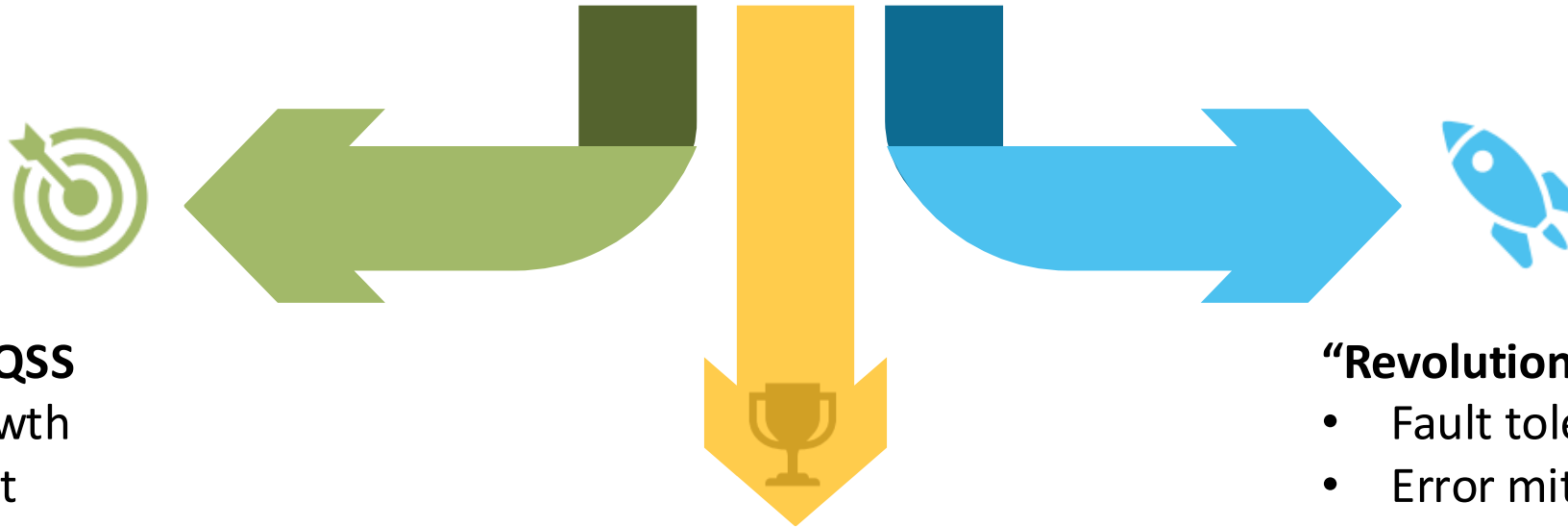
>>> backend = IQMAuroraBackend()

>>> qc_transpiled = transpile(qc, backend)

>>> job = backend.run(
    run_input=qc_transpiled, shots=args.shots)

>>> print(job.result().get_counts())
```

Looking Forward: Where Next for MQSS ...



Stabilization of MQSS

- Continued Growth
- CI & CD support
- User Interfaces
- Transfer to Operation

“Evolution” of MQSS

- New backends/systems
- Better optimizations
- Easy-to-use abstractions
- User support components

“Revolution” of MQSS

- Fault tolerance
- Error mitigation
- Basic research work
- Crosscut for entire MQSS

Enabling Quantum Acceleration in Hybrid HPCQC Workflows for and beyond NISQ